Vinay Traders

Frontend to Backend Handoff

Routes, data contracts, database mapping, API connection, and server-side business rules

Application	Vinay Traders back-office and POS
Frontend	React + TypeScript + Vite
Recommended backend	PostgreSQL / Supabase with authenticated API access
Purpose	Give the backend team the exact contract needed to connect the finished screens

Prepared for backend implementation

Vinay Traders frontend handoff

This document describes the completed frontend, the data contract it currently uses, and the backend work required to connect it to a real database and API.

What is already complete

The frontend is a React + TypeScript + Vite application. It currently runs with seeded data and persists changes in browser `localStorage` through `src/services/data.ts`. All business mutations are centralized in `src/services/operations.ts`, so the backend integration should preserve those input and output shapes.

The backend team should replace the local data adapter with authenticated API calls. The UI routes, components, calculations, and form behavior can stay in place.

Frontend file structure

```
vinay-traders/
■■ public/
■  ■■ favicon.svg
■  ■■ icons.svg
■■ src/
■  ■■ assets/
■  ■  ■■ hero.png
■  ■  ■■ react.svg
■  ■  ■■ vite.svg
■  ■■ components/
■  ■  ■■ AppErrorBoundary.tsx       # Application-level error fallback
■  ■  ■■ DataTable.tsx              # Searchable, paginated table
■  ■  ■■ ui.tsx                     # Button, Card, Modal, Field, EmptyState, etc.
■  ■■ features/
■  ■  ■■ attendance/
■  ■  ■  ■■ index.tsx               # Monthly attendance route export
■  ■  ■■ backoffice/
■  ■  ■  ■■ index.tsx               # Dashboard operations: inventory, purchase,
■  ■  ■                             # suppliers, workers, settings, deposits
■  ■  ■■ billing/
■  ■  ■  ■■ commerce.css           # POS and billing styles
■  ■  ■  ■■ index.tsx               # Billing, invoice list, invoice paper
■  ■  ■  ■■ parts.tsx               # Product search, line editor, payment dialog
■  ■  ■  ■■ store.ts                # Held bill local state
■  ■  ■■ customers/
■  ■  ■  ■■ index.tsx               # Customer list, detail, forms, payments
■  ■  ■  ■■ people.css
■  ■  ■  ■■ shared.tsx              # Shared people form/payment components
■  ■  ■■ plumbers/
■  ■  ■  ■■ index.tsx               # Plumber list, detail, commission, payments
■  ■  ■■ workers/
■  ■     ■■ index.tsx              # Worker detail, payroll, monthly attendance
■  ■     ■■ workers.css
■  ■■ lib/
■  ■  ■■ cn.ts                     # Class name helper
■  ■  ■■ supabase.ts                # Existing Supabase client scaffold
■  ■■ mock-data/
■  ■  ■■ index.ts                  # Seed data for local development/demo
■  ■■ services/
■  ■  ■■ data.ts                    # CURRENT localStorage/query adapter
■  ■  ■■ operations.ts              # CURRENT business mutation rules
■  ■■ store/
■  ■  ■■ ui.ts                      # Sidebar and command palette UI state
■  ■■ utils/
■  ■  ■■ commission.ts              # Plumber commission calculation
■  ■  ■■ format.ts                  # INR/date/ID helpers
■  ■  ■■ gst.ts                     # GST helpers
■  ■  ■■ inventory.ts               # Stock/value/status helpers
■  ■  ■■ invoice.ts                 # Invoice totals and payment status
■  ■  ■■ payroll.ts                 # Worker payroll calculation
■  ■  ■■ units.ts                   # Unit conversion helpers
■  ■■ App.tsx                       # Shell and route definitions
■  ■■ App.css                       # Application layout styles
■  ■■ index.css                     # Design tokens and global styles
■  ■■ main.tsx                      # React, router, query provider bootstrap
■■ supabase/
■  ■■ config.toml
■  ■■ migrations/                   # Existing products-only starter migration
```

```
■■ package.json
■■ vite.config.ts
■■ tsconfig*.json
■■ wrangler.jsonc
```

Frontend routes and backend resources

Frontend route	Screen	Backend resources needed
`/` and `/dashboard`	Dashboard	sales, purchases, customers, products, plumbers, deposits
`/billing`	POS checkout and invoice history	products, customers, plumbers, invoices, invoice items, payments
`/billing/invoices/:id` and `/billing/:id`	Invoice detail/print	invoice, invoice items, customer, plumber, payments, business settings
`/purchase` and `/purchase/new`	Purchase register/editor	suppliers, products, purchases, purchase items, payments, stock
`/purchase/:id`	Purchase detail	purchase, purchase items, supplier, payments
`/inventory` and `/inventory/new`	Product catalogue	products, variants, batches, suppliers
`/inventory/products/new`	Product creation alias	products, variants, batches
`/inventory/products/:id`	Product detail	product, supplier, stock transactions, batches, price history
`/inventory/ledger` and `/inventory/stock-ledger`	Stock ledger	stock transactions, products, users
`/inventory/price-matrix` and `/inventory/pricing`	Bulk price updates	products, price history
`/customers`, `/customers/new`, `/customers/:id`	Customer management	customers, invoices, payments, plumbers
`/plumbers`, `/plumbers/new`, `/plumbers/:id`	Plumber/referral management	plumbers, customers, invoices, payments
`/workers`, `/workers/new`, `/workers/:id`	Worker management	workers, attendance, payments
`/workers/payroll`	Payroll register	workers, attendance, worker payments
`/attendance` and `/attendance/monthly`	Attendance matrix	workers, attendance
`/patsanstha`	Daily deposit ledger	deposits, users
`/settings/*`	Business, billing, inventory, commission, deposits, workers, roles	settings, role permissions
`/notifications`	Notification inbox	notifications

Current TypeScript data contract

The canonical TypeScript interfaces are in `src/types/index.ts`. The backend should expose JSON with the same property names and ISO date values. IDs are strings in the frontend; UUIDs are recommended in the database.

Core entities

- `Product`: catalogue and sellable variant data, pricing, GST, stock summary, location, supplier, and `batches[]`.
- `Supplier`: supplier contact, GSTIN, address, and current outstanding balance.
- `Customer`: customer contact, type, credit limit/days, optional plumber referral, balance, purchase summary.
- `Plumber`: referral contact, commission rate, referral totals, and commission paid.
- `Worker`: monthly fixed or daily wage worker details and payroll adjustments.
- `Attendance`: one worker/date record with status, check-in/out, overtime, and notes.
- `Invoice`: sale header, customer/plumber, line items, totals, payment status, commission, and lifecycle status.

- `Purchase`: supplier invoice header, line items, totals, payment status, dates, attachment name, and lifecycle status.
- `Payment`: account payment or worker adjustment. `entityType` is `customer`, `supplier`, `plumber`, or `worker`.
- `Deposit`: daily Pat Sanstha expected/paid amount, status, receipt, and attachment.
- `StockTransaction`: immutable stock movement with reference, in/out quantity, balance, location, and user.
- `PriceHistory`: rate change audit record.
- `Settings`: business profile, GST, invoice numbering, commission, payroll, deposit, language/theme, and permissions.
- `AppNotification`: notification inbox item with destination route and severity.

Important enums

```
PaymentMethod: Cash | UPI | Card | Bank | Credit | Mixed
PaymentStatus: Paid | Partial | Pending | Overdue | Credit | Refunded
InvoiceStatus: Draft | Held | Completed
PurchaseStatus: Draft | Completed
AttendanceStatus: Present | Absent | Half Day | Paid Leave | Unpaid Leave | Weekly Off
CustomerType: Walk-in | Regular | Business | Contractor
```

Recommended database structure

Use PostgreSQL/Supabase or an equivalent relational database. Keep monetary columns as `numeric(12,2)` and quantities as `numeric(14,4)`. Never use floating point for persisted money.

```
profiles / users
business_settings
role_permissions

products
product_variants                 # optional if each product row is already a variant
inventory_batches
stock_transactions
price_history

suppliers
customers
plumbers
workers
attendance

invoices
invoice_items
purchases
purchase_items
payments
payment_allocations               # recommended for applying one payment to many bills

deposits
notifications
```

Table mapping

Frontend object	Recommended table(s)	Notes
`Product`	`products`, `inventory_batches`	Keep `stock`, `reserved`, and `conversion_factor` server-maintained. Store images separately if uploads are required.
`Supplier`	`suppliers`	Do not accept `balance` as an authoritative client value; derive it from purchases and payments or maintain it transactionally.
`Customer`	`customers`	`balance`, `total_purchase`, `bills`, and `last_purchase` are projections/cache fields.
`Plumber`	`plumbers`	`commission_earned` and `commission_paid` should be derived from completed invoices and payments.

Frontend object	Recommended table(s)	Notes
`Worker`	`workers`	Store baseline bonus/advance/deduction with a period or move adjustments to a worker adjustment table.
`Attendance`	`attendance`	Add a unique constraint on `(worker_id, date)`.
`Invoice`	`invoices`, `invoice_items`	Header and lines must be saved atomically. Totals are recalculated on the server.
`Purchase`	`purchases`, `purchase_items`	Unique completed supplier invoice per supplier: `(supplier_id, supplier_invoice)`.
`Payment`	`payments`, `payment_allocations`	Keep a payment ledger; allocation rows update individual invoice/purchase dues.
`Deposit`	`deposits`	Add a unique constraint on business/location and date.
`StockTransaction`	`stock_transactions`	Append-only audit table. Never edit a posted movement; create a correction movement.
`PriceHistory`	`price_history`	Append-only rate audit.
`Settings`	`business_settings`, `role_permissions`	Scope settings by business and, if needed, branch/location.
`AppNotification`	`notifications`	Scope by user and business.

API connection contract

The simplest integration is to keep the existing `useData()` return shape and change its implementation from local storage to an API client.

Recommended environment variables:

```
VITE_API_BASE_URL=https://api.example.com/v1
VITE_SUPABASE_URL=...
VITE_SUPABASE_PUBLISHABLE_KEY=...
```

Recommended read endpoints:

```
GET  /bootstrap                 # returns the AppData-shaped dashboard payload
GET  /products
GET  /products/:id
GET  /suppliers
GET  /customers
GET  /customers/:id
GET  /plumbers
GET  /plumbers/:id
GET  /workers
GET  /workers/:id
GET  /attendance?month=YYYY-MM
GET  /invoices?status=&from=&to=&customer_id=
GET  /invoices/:id
GET  /purchases?status=&from=&to=&supplier_id=
GET  /purchases/:id
GET  /stock-transactions?product_id=&from=&to=
GET  /price-history?product_id=
GET  /deposits?from=&to=
GET  /notifications
GET  /settings
```

Recommended write endpoints:

```
POST  /invoices                   # create draft or completed sale
PATCH /invoices/:id               # edit a draft only
POST  /invoices/:id/complete      # atomically validate payment and deduct stock
POST  /invoices/:id/hold

POST  /purchases                   # create draft or completed purchase
PATCH /purchases/:id              # edit a draft only
POST  /purchases/:id/complete      # atomically validate payment and add stock

POST  /products
```

```
PATCH /products/:id
POST  /products/:id/stock-adjustments
POST  /price-matrix                # bulk price update

POST  /customers
PATCH /customers/:id
POST  /plumbers
PATCH /plumbers/:id
POST  /workers
PATCH /workers/:id
PUT   /attendance/:worker_id/:date
POST  /payments                    # customer, supplier, plumber, worker
POST  /deposits
PATCH /settings
POST  /notifications/:id/read
```

Every mutation should return the updated resource or the updated aggregate needed by the UI. For the initial integration, returning an `AppData`-shaped `/bootstrap` response after mutation is acceptable; later, switch to targeted query invalidation.

Business rules the backend must own

The current frontend implements these rules for demo data. The backend must enforce them again because browser validation is not security:

1. A completed sale validates active products, allowed units, minimum selling price, GST, payment method, customer credit limit, and available stock. 2. Completing a sale deducts stock, consumes the selected batch when supplied, appends sale stock transactions, updates customer balance, and records plumber commission in one database transaction. 3. A completed purchase validates the supplier invoice, adds stock/batches, updates supplier balance, appends purchase stock transactions, and records price history in one transaction. 4. Draft invoices and purchases must not affect stock, balances, commission, or payment ledgers. 5. A credit bill cannot contain a paid amount. Mixed payment requires at least two distinct methods, and payment split totals must equal the paid amount. 6. Payment posting allocates money to the oldest outstanding bills or uses explicit allocation rows. It must never make a customer/supplier/worker balance negative by accident. 7. Attendance is unique per worker/date. Payroll is calculated for a selected `YYYY-MM` period from attendance and worker adjustments. 8. Stock transactions and price history are append-only audit records. 9. Invoice and purchase numbering must be generated on the server with a transaction-safe counter per business/branch. 10. All list and detail endpoints enforce tenant/business scope and role permissions.

Suggested response shapes

Error response:

```
{
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "message": "Insufficient stock for CPVC Pipe 1 inch.",
    "fields": {"items[0].quantity": "Available quantity is 8."}
  }
}
```

Invoice create request (the frontend already sends the equivalent of this object):

```
{
  "customerId": "cust-001",
  "plumberId": "plumber-001",
  "items": [
    {
      "productId": "p-cpvc-1",
      "name": "CPVC Pipe",
      "variant": "1 inch · 3 Meter",
      "hsn": "3917",
      "quantity": 2,
      "unit": "Meter",
      "rate": 92,
      "discount": 0,
      "gst": 18,
      "batchId": "batch-001"
    }
  ],
```

```
  "paid": 217.12,
  "paymentMethod": "UPI",
  "status": "Completed",
  "interstate": false,
  "notes": ""
}
```

The server response should include the generated invoice number, server-calculated totals, payment status, and updated stock/balance projections.

Integration sequence

1. Backend creates the schema, authentication, business/branch scope, and seed/reference data. 2. Backend implements `GET /bootstrap` using the frontend property names. 3. Frontend team replaces `getData/saveData` in `src/services/data.ts` with API reads and cache invalidation. Do not change every screen individually. 4. Backend implements invoice and purchase completion transactions first, then payments, stock adjustments, attendance/payroll, deposits, and settings. 5. Add integration tests for duplicate supplier invoices, insufficient stock, credit limits, mixed payments, payment allocation, payroll periods, and stock ledger balances. 6. Remove local demo persistence only after the authenticated API path has passed the route smoke tests.

Current local/demo limitation

`src/lib/supabase.ts` is only a client scaffold and the existing Supabase migrations cover a products table plus a temporary public read policy. It is not the completed production schema. The backend team must add authentication, tenant/branch scoping, all tables above, server-side rules, and restrictive RLS policies before production use.